

Systolic Array Matrix Multiplication Accelerator (SAMMA) - Pipette Version

By: Vance Borus, Sachal Shaikh, Byeongguk Lee, Chenyi Wang, Neal Causey, and Sean Bubernak.

Introduction	2
Overview	2
Related Work	3
Learning	3
Industry Standard	4
Planning	6
Design Specification	6
Collaboration	7
Timeline and Milestones	10
Design	11
RTL	11
Verification	12
Physical Design	13
Optimization	14
Evaluation Results	15
Layout Photo	15
Quality Metrics	16
Analyses	18
References	19

Introduction

Overview

Systolic arrays are at the heart of any TPU as they have the advantage of prioritizing data reuse and thus minimizing memory accesses. Traditionally, TPUs use a simple weight stationary systolic array to achieve this. However this comes with its own problems. In order to compute in a systolic fashion, the passed in activation and weight matrices must be shifted before they get funneled in as well as being shifted again on the way out. This shifting requires the usage of many shift registers that both take up a non-trivial amount of space in addition to adding to the overall power draw of the chip. It also requires more cycles to load in matrices due to this shifting. Our new pipette architecture of the systolic array eliminates the need for these shift registers, thus reducing the power consumption, area, and latency when loading.

To transition from a theoretical concept to a viable real-world solution, several realistic factors must be explicitly considered.

Global Factors: Because we are sending the chip to TSMC to be fabricated. We want to actually be studying something meaningful. Once we get the chips back (Vanilla and Pipette) we will be able to actually test the two via an FPGA and see the PPA differences outside of those seen in the generated reports.

Economic Factors: Our chip is only 2mm x 2mm in area (despite using a 180nm process node) so a primary challenge that we encountered was working at such an area constraint. Larger dies significantly increase manufacturing costs due to higher defect probabilities. Pipette's layout has major improvements in terms of the physical footprint of the systolic array. This became especially apparent when floorplanning as the Pipette team was able to fit their design without too much difficulty while Vanilla was having concerns about if they would actually be able to fit theirs at all. Beyond core logic, commercial SRAM macros and high-speed I/O interfaces (like PCIe) carry massive licensing costs [1]. To avoid these steep non-recurring engineering costs and bypass complex mixed-signal design validation, we also omitted an on-chip PLL [2]. Relying on an external clock routed via a digital I/O pin saved crucial silicon area, maximizing available real estate strictly for our core systolic array logic.

Environmental Factors: Due to both the elimination of the shift registers as well as optimizing data reuse within the systolic array, we were able to see major gains in terms of operational power consumption reductions. This directly reduces the carbon footprint and thermal dissipation demands when scaled across large-scale data centers.

Related Work

Classical systolic arrays have remained a dominant architecture for matrix-multiplication acceleration since Kung's foundational work [6], owing to their high compute density and local data reuse that minimizes bandwidth requirement. This principle was commercialized at scale by Google's TPU v1 [5], which mainly adopts weight-stationary dataflow, weights reside in the PEs while activations stream in horizontally and partial sums accumulate vertically. However, classical implementations depend on skew which means operand streams must be staggered at the array boundary so that data align correctly inside the mesh. This alignment requires $O(D^2)$ shift registers, introducing additional area and power overhead as well as a $D-1$ cycle tail latency on each GEMM.

Gemmini [7], in contrast, is a full-stack, open-source generator that produces classical systolic-array accelerators with mature, parameterizable Chisel implementations of the PE, the systolic mesh, and the surrounding controller. This makes it a valuable reference point for a tape-out project: its well-tested RTL provides a baseline PE datapath and mesh organization that a hardware prototype can build upon or compare against. However, the array it generates remains a conventional WS/OS mesh that still carries the boundary shift-register and alignment logic our Pipette architecture is designed to remove.

The most closely related work is DiP [4], which explicitly proposes a diagonal-input, permuted weight-stationary dataflow to remove skew. Our design shares the same motivation of eliminating boundary shift registers. However, DiP relies on an offline pre-permuted weight layout, whereas our Pipette architecture generates the required row-rotated layout directly through in-place weight loading. This preserves the canonical boundary data format and avoids any topology-specific scratchpad contents or dedicated weight-transform hardware, which achieve the drop-in replacement of the Vanilla design.

Prior work either focuses on general dataflow optimization [7] or explores skew elimination at the logical level [4]. The objective of this tape-out is to demonstrate the feasibility of the skew-free Pipette topology under real silicon fabrication (TSMC 180nm) and to enable a measured PPA comparison against a conventional Vanilla architecture, filling the gap of whether such a topology can be physically realized under hard constraints.

Learning

Completing this project involved encountering many challenges, ones that we expected to encounter as well as ones that we could not have anticipated. These are detailed in the below sections.

Toolchain and Configuration Management: Early in the physical implementation phase, we encountered friction with the automated build system. Not only was using a makefile

still somewhat unfamiliar for some of us but it was made more difficult by strict conventions such as requiring all source files listed in `src.yml` to carry a `.v` extension. Ensuring we got what we wanted out of make often required a deep dive into the build infrastructure to ensure predictable compilation and synthesis runs.

Front-End Hardware and Control Logic: Designing the control logic to properly initialize the system presented a timing challenge. Generating and synchronizing the very particular reset sequence that `bsg_link` uses was non-trivial and doing so externally required careful coordination to ensure that all internal registers, control pipelines, and accumulation buffers settled into a known, valid state before the first matrix data waves were injected.

Interface Bit-Width Constraints: The `bsg_link` IP architecture, either purposely or not, imposed a structural constraint that we did not know going in, it was incapable of handling odd data widths. This became apparent to us when we decided to tack on a parity bit onto our data flits, (more on this below). This forced us to carefully evaluate our data packet sizing and alignment strategies at the boundary interface to remain compliant with link's hardware limitations.

Error Detection and Interface Pin-Count Trade-offs: When we first approached the problem of adding parity to our data flights, we immediately jumped to simply allocating another bit to both the sides of our data flit IOs to check/generate parity then. However, later we realized that we had never even considered simply sending the parity in another flit altogether. This would eliminate the need to use 17 pins for each side of the IO, as the data flits could contain exclusively data, only requiring 16 pins for both input and output. Considering that at flit would still be the same width as for the data flits though, the other advantage to this is that with that additional space, we would be able to do error checking and correction (ECC) to actually fix the bit flips as they come up, not just identify when they occurred. Although in the end, we had a few more IOs then we needed so packing the parity into the data flits would save us from the latency hit that would come from sending additional exclusive parity flits in addition to our data flits.

While the technical and collaborative challenges highlighted above represent our most significant takeaways, this project ultimately facilitated the growth of many major engineering and teamwork skills across our group. The experience of managing this design from architectural concept to physical implementation has fundamentally absolutely elevated our capabilities as engineers.

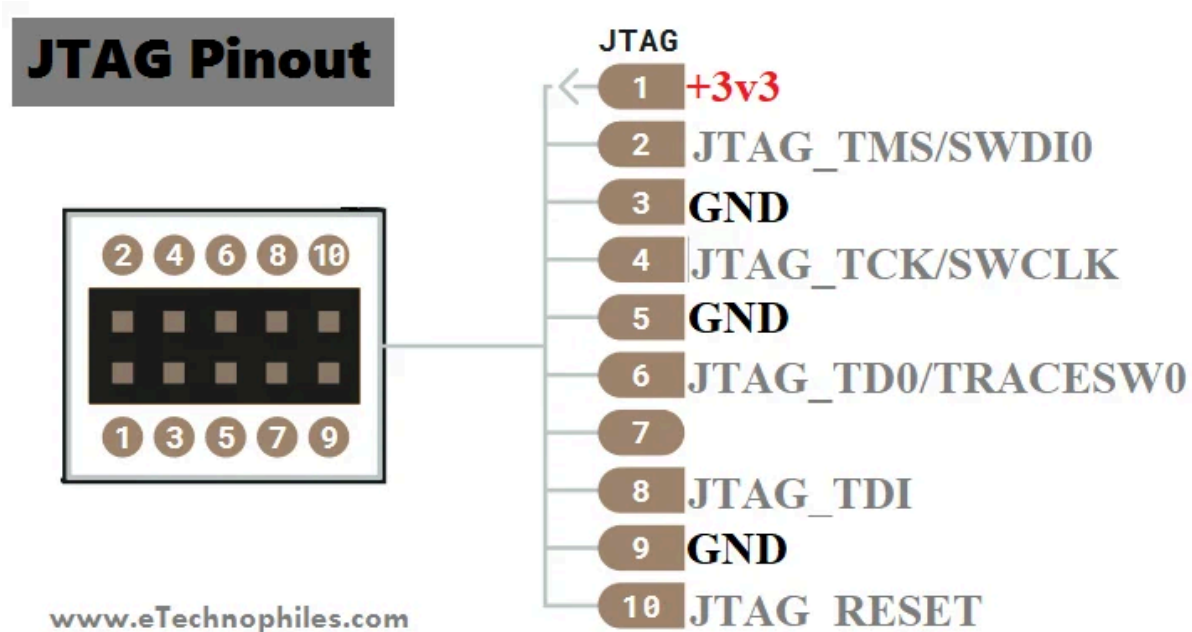
Industry Standard

Our chip design is a research prototype designed to explore the benefits of the Twisted Torus architecture over the standard Vanilla architecture. Because it is a prototype, an interface

for internal debugging is essential for analyzing the chip and performing real-world PPA comparisons, as well as ensuring correct functionality of the internal components. To accomplish this we needed a suitable protocol for conducting internal debugging. The most common interface is JTAG, described by IEEE 1149.1 [3].

JTAG has four data wires, a reset wire, and power/ground, see Figure 1. It incorporates FSM logic which allows for active-chip debugging in live environments, with support for ATPG (Automatic Test Pattern Generation), issuing of instructions, flash programming, and boundary scanning. While a comprehensive feature-set and industry proven, the JTAG interface IP takes up valuable chip area, and not an insignificant amount. Additionally, JTAG adds features that are not applicable to our chip, such as program flashing.

Figure 1 - JTAG 7 Wire Interface [9]



Because JTAG proved to be too much for our simpler chip design, we explored another protocol, SPI [8]. SPI also uses a four wire interface, with the advantage of fast duplex communication. While it allows fast streaming of I/O data, for a debug interface it is excessive. It also requires additional chip area to be taken up to support the SPI controller.

While both of the protocols we explored had advantages, our very limited chip area required us to look at a very simple alternative. The alternative we settled is called a Scan Chain. A scan chain works by replacing normal DFFs in the chip with Scan Chain DFFs, which have an extra enable signal. When this enable signal is on, the SCDFE will shift its data to the next SCDFE in line, and all of the SCDFEs will be connected into one long shift register. This shift

register interface will then be exposed to three top level wires, Scan In, Scan Out, and Scan Enable.

The Scan Chain requires no additional logic to implement. In addition, the synthesis tool can perform automatic DFT and Scan Chain insertion, allowing us to write RTL with no restrictions, then include specific modules to be added to the Scan Chain. Each Scan Chain DFF is about 1.5x the size of a normal DFF. The raw Scan Chain also allows us to perform ATPG, generating a chip stimulus pattern to test for stuck-at-1 and stuck-at-0 errors from manufacturing.

Planning

Design Specification

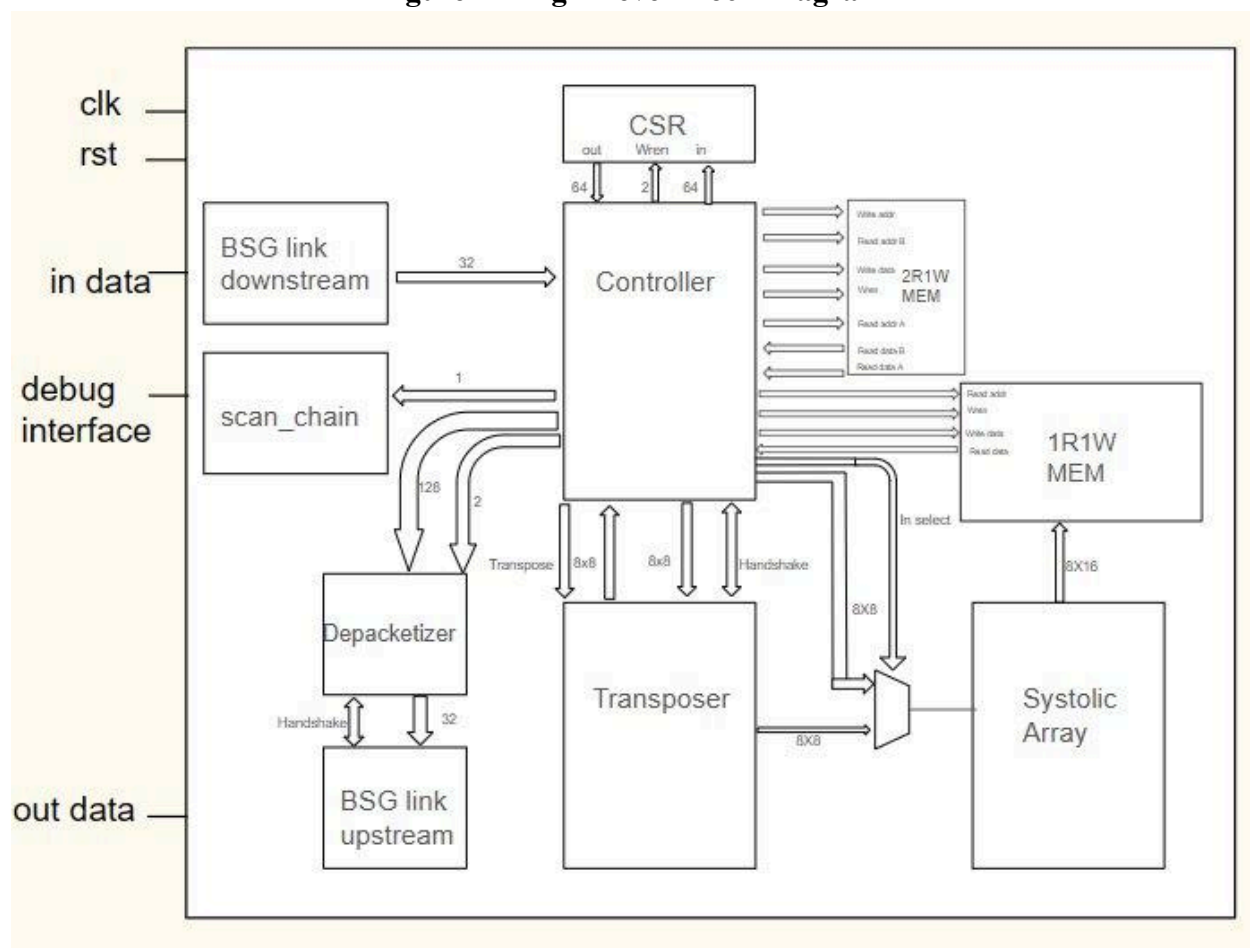
Our project had the following general specifications.

- TSMC 180nm process node.
- 2mm x 2mm die, with 1.7mm x 1.7mm available for core logic.
- 64 I/O pins, 16 reserved for power and ground.
- Less than 1 million total transistors.
- 50 MHz target clock frequency.
- No SRAM compiler required.
- No PLL clock generator, external I/O clock used instead.

Table 1 - Chip Success Standards

Metric	Minimum	Ideal	Exceptional
Clock speed	Less than 50 MHz	About 50 MHz.	Greater than 50 MHz
I/O usage	Limited to 8 bit input or 8 bit output.	16 bit input and output.	Extra bits for parity or expanded I/O functionality
Systolic Array	INT8 input, INT16 results	INT8 input, INT32 results	INT16 input INT32 results
Performance	Performs correctly, but takes longer than expected, and is unstable at higher speeds.	Performs correctly and is stable.	Performs faster than expected and is exceptionally stable.
Power	200 mW	160 mW	120 mW
Area	75% density.	80% density.	85% density.
Twisted Torus Functionality	Functional but is limited due to constraints, and is worse than vanilla.	Torus is functional, and matches vanilla.	Torus works and performs better than vanilla.
DRC/LVS	No errors but sections had to be disabled to pass.	No errors but constraints had to be relaxed to pass.	No errors and design was not limited by DRC/LVS errors.

Memory	Had to be set to the minimum to meet area constraints, limiting functionality	Set at a modest size, able to perform full throughput operations.	Extra memory space added due to area availability.
Debug Interface Functionality	Scan-chain logic had to omit some sections to save area	All important components are along the scan-chain.	In addition to the scan-chain, an additional interface was added.
Chicken Bits	No chicken bits were set.	A couple chicken bits are usable for chip configuration.	Several chicken bits are set for various modes of operation.

Figure 2 - High-Level Block Diagram

High level inputs: clock, reset, debug inputs, in-data (16 bits at DDR = 32 bits per cycle), ready/valid

High level outputs: debug outputs (16 bits at DDR = 32 bits per cycle), ready/valid

Collaboration

Our team consisted of six people who each had different roles and responsibilities throughout the project. Several categories of the project required sequential completion, so

members would often be responsible for different phases of the project as the design progressed. In Table 2 below, we can see each member of the team and their contributions at each phase of the project.

RTL refers to the SystemVerilog designing of modules that make up the chip design. This was the biggest step in the project. In Table 3 we can see how each member contributed to the individual modules both in terms of writing them, and of testing them. Almost all of the team were responsible for at least one module.

Verification was the process of writing modules tests, and conducting simulations to ensure that the chip functions correctly. Incorrect chip behavior resulted in issuing revisions to fix observed issues, then re-simulating to test issued corrections. We had a dedicated member responsible for chip testing, Sachal, with assistance from Brian.

Physical Design involved pushing our chip design through the physical design flow, first synthesis then place-and-route. Doing so introduced challenges with chip area, timing violations, and DRC/LVS violations. All of which would need to be addressed. Brian was the majority contributor, with assistance from Vance.

DRC/LVS refers to issues with the physical design that violated either manufacturer set rules, or a physical design that does not match the code. DRC issues often required manual fixing, which Brian took care of as part of physical design. LVS issues would require a re-run of the physical flow with new chip constraints.

Signoff Verification entails running a more rigorous test on a release candidate of the chip for final verification. If these pass, then the chip is good to go. Brian also handled these as part of the physical design flow.

Document Generation was a team effort. In conjunction with the Vanilla team, a document that set the ISA spec was written, as well as the proposal for the chip design as a whole. Documentation also includes the Apple design review conducted to allow Apple engineers a chance to evaluate our design.

ENGINE was an event slated to allow teams to showcase their chip designs to a convention of attendees, and required several materials to be procured for the event. Vance was the primary contributor here, with assistance from Sachal, Sean, and Chenyi. The ENGINE showcase was held in a combination with the Vanilla team, whose contributions are not mentioned here.

Table 2 - Contributions in Project Categories by Team Member

Category	Chenyi	Brian	Sachal	Sean	Vance	Neal
RTL						
Verification						
Physical Design						
DRC/LVS						
Signoff Verification						
Document Generation						
ENGINE						

Member contributions broken up by category of project work.

Table 3 - Module Contribution Breakdown by Team Member

Submodule	Chenyi	Brian	Sachal	Sean	Vance	Neal
PE						
TwistMesh						
Transposer						
Depacketizer						
bsg_link						
Scratchpad Memory						
Controller						
CSR						
Top Level						

Blue: Tested Only. Orange: Contributed Only. Green: Tested and Contributed. White: Did not contribute.

RTL module design and testing contributions broken up by team member.

To organize our team, we set up a Github page to manage RTL code and other shared files, and keep track of versions and issue changes for verification. This system allowed us to streamline the development process of RTL in the early stages of the chip design, and make the verification process easier during later stages. For communication, we set up a shared Discord

messaging server, which allows separate channels to easily organize communication topics, and voice channels for meetings or discussions. This server was also shared with Vanilla, allowing easy inter-team communication for shared resources.

Timeline and Milestones

This section summarizes the team's weekly progress throughout the quarter and compares the project timeline against the major design milestones. In Week 1, the team established the project file structure and began setting up the TSMC 180 nm library and design environment. Week 2 focused on RTL development, including refinement of the Processing Element (PE) and TwistMesh modules. During this week, the Transposer module from EE 477 was also reviewed and largely carried forward requiring some redesign.

In Week 3, testing began for the PE, TwistMesh, and Transposer modules. This was also the week when the team gained access to the TSMC 180 nm library, allowing us to begin viewing and evaluating early physical design results. In Week 4, development began on the Depacketizer module, and the team also implemented and tested `bsg_link` for chip communication. In Week 5, work began on the Scratchpad Memory while previously completed modules started to be frozen.

Weeks 6 and 7 focused on the RTL design and testing of the Scratchpad Memory and Controller modules. These were completed in Week 7 alongside the creation of the Control and Status Register (CSR) module. Week 7 was also a major project milestone because the team participated in the Apple design review, where members of Apple's silicon physical design verification team reviewed the project and provided feedback.

In Week 8, top-level integration and testing began, with most lower-level modules largely frozen. This week also marked the RTL freeze milestone. Week 9 was dedicated primarily to full-design verification, including testing the integrated design and resolving remaining functional issues. Week 10 focused on physical verification, final design cleanup, and preparation of the design files for submission.

Overall, the team followed a staged development process moving from sub module-level RTL, to sub module testing, to top-level integration, verification, and final physical design preparation. The major milestones were the Week 7 Apple design review, the Week 8 RTL freeze, and the Week 10 final physical verification and submission preparation.

Design

RTL

The RTL portion of this project implements a Twist systolic array together with the surrounding control, memory, and I/O infrastructure required to run it as a self-contained accelerator. All modules are written in SystemVerilog following BaseJump STL conventions. The compute units are functionally equivalent to the Gemmini [7] Chisel reference, while the rest of the system is built from scratch to expose a clean instruction-driven interface. The design is fully parameterized on the mesh dimension, which must be a power of two, so the same RTL targets a smaller mesh during functional bring-up and a larger mesh for the final PPA comparison without code changes.

At the lowest level, the processing element contains exactly five registers: two weight double-buffers, a horizontal weight relay, and two one-cycle control pipelines for lock and propagate. The MAC operates on the active buffer selected by the incoming propagate signal, producing a signed wide-accumulator psum with saturation. A new weight is captured into the inactive buffer on the rising edge of the lock signal, which guarantees a single capture per traveling lock pulse. The ifmap and psum outputs are combinational, while weight, lock, and propagate are registered inside the PE. A global valid signal gates every register so that the pipeline can stall and drain cleanly.

The mesh layer arranges the PEs into the twisted-torus structure with two named interconnects. A horizontal path carries weights and partial sums through PEs in the same row, while a diagonal path carries ifmap and lock with a row wrap-around. Pipeline registers for ifmap and psum sit between PEs and are gated by a per-column valid shift register, which also drives the column-wide valid into the PEs. Because the torus diagonal already aligns the results by row, the mesh reads outputs directly from the last column without any de-braiding network.

The control path covers instruction decoding, queuing, dispatch, and four execution workers. Incoming flits are parsed by a multi-state decoder that handles single-flit headers and multi-flit immediate payloads, queued in a small command FIFO, and routed by a dispatch stage to the write, read, execute, or CSR worker based on opcode. The compute worker is a multi-state FSM that pre-primed the transposer when needed and then hands a mesh request to the mesh driver. The mesh driver sequences a fire / drain / finish window, asserts the per-row lock exactly when the corresponding weight column arrives, and flips a global propagate bit only on a compute instruction that follows a load.

Memory is provided by three banks (ifmap, weight, psum) built on a register file which functionally similar to 1R1W SRAM macro. The top rows of each bank are reserved as identity

or zero ROMs, used for utility paths and for bias clearing. A memory arbiter switches the banks between the host write/read ports and the executing engine, signaling a conflict when both attempt access. A small parameterized transposer provides row/column reordering and is shared between compute (ifmap or weight transposition) and the standalone transpose instruction.

The chip wrapper integrates a 48-pad ring, a 16-bit DDR source-synchronous link with parity protection, and a staged reset sequencer around the instruction-driven SoC core. Outbound traffic is multiplexed across read, write, execute, and CSR result paths and serialized into flits for the upstream link. Status and error events are aggregated by a CSR router and stored in a 64-bit CSR with read/set/clear/assign semantics, providing host visibility into overflow, ROM-write, decoder, arbiter, transpose-conflict, and parity events.

Verification

Verification for the chip was done using the trace replay modules provided by Basejump STL. Because we had to measure full throughput and functionality for a high level design we modified our testbenches to instantiate two trace replays; one for the send side and one for the receive side. We also implemented a valid-then-ready handshake for the top level testbench, which tested the functional chip as well as simulating bsg link on an FPGA for full throughput test cases.

Individual module's testbenches and traces were written to test functionality and edge cases. At the top level we wrote four benchmarks for our chip to run. We wanted to write things in such a way that someone could write instructions in our ISA language and then directly test this on a chip. To do this we wrote a python script to a) convert instructions to machine code, b) write send traces for the chip (by adding the send command and attaching a NOOP command for NOOP instructions), and c) write receive traces for the chip (by receiving header flits and expected output data).

Benchmark 1 tests general functionality of the chip by writing a few matrices, doing a few computations, and reading their results. Benchmark 2 tests matrix multiplication holistically, including edge cases (multiplying by zero, identity, and overflow matrices) and random matrices. Benchmark 3 tests every possible instruction. Benchmark 4 tests full throughput and invalid memory accesses.

Aside from trace replay telling us where benchmarks failed, we used Verdi, see Figure 3, to look at internal signals to diagnose and debug issues. Most debugging was done in sim-rtl, and after that sim-syn, sim-par, and formal-syn confirmed the chip worked.

Figure 3 - Verdi Simulation Waveform of Working Chip

Physical Design

The physical design portion of the flow involved primarily the Place and Route tool (PAR), which handled automatic placement of gates and routing nets to implement the logic described in RTL, and the netlist generated by the Synthesis (SYN) tool. While automatic, it would often struggle to avoid illegal routing in particularly dense areas of the chip. To remedy this, we implement a manual floorplan, shown below in Figure 4.

Floorplanning is the act of helping out the placement tool by creating either guides, fences, or regions when a specific module should go, and at what density it should attempt to achieve. Guides are the weakest, and are more of a suggestion to the tool to try and place the majority of the gates within the boundaries. We exclusively used guides as it gave the tool the most flexibility. Fences restrict the module to within the dedicated area, however any unused area can allow other modules to extend into the fenced area if needed. Region is the most strict, restricting the module to only within the boundaries like a fence, but also excluding other modules from using unused space within the Region.

When choosing a module's floorplan, two things need to be considered: location and density. Knowing how the modules connect to each other allows us to place modules with these connections next to each other, simplifying routing. Since we also interface with the padding which contains our I/O pads, modules that connect to these pads should also be placed near their respective pads. For density, the important limitation there is hold timing. Hold timing is dependent on the fastest path in a chip. If a path is too fast, it will violate hold timing. The highest risk for this is in places where DFFs have little to no gates between different DFFs, such as our memory array. To combat this, the tool will insert buffer gates to add path delay, however this requires space. Lowering the density of high-DFF areas can allow the insertion of more buffers, and close the gap of hold timing violations.

The final aspect of physical design is DRC and LVS violations. DRC errors occur when the tool places metal layers wrongly, and these require manual fixing. We were able to reduce our DRC errors down to a single error, which was a metal fill error. Metal fill is when a given area of the chip is absent of enough metal on a certain layer, so all that is needed to fix is to draw extra metal in the missing area and it is fixed. LVS errors are usually a product of a design that is too dense, so improved floorplanning will typically fix these issues.

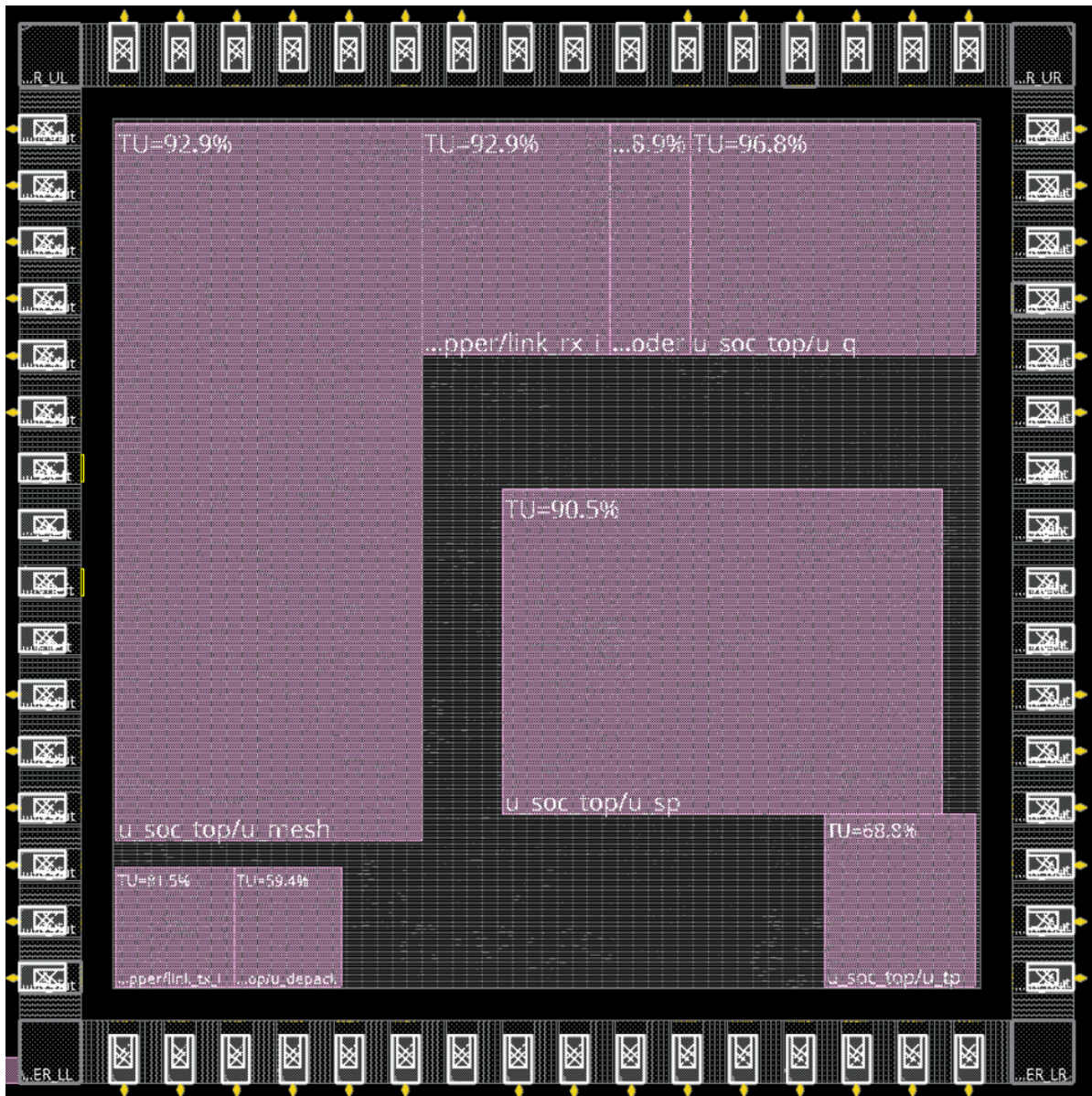
Figure 4 - Innovus Floorplan of Top Level Modules

Figure detailing the Floorplaning mode of Cadence Innovus, and our floorplan which was used to help the PAR tool with module placement to optimize for area usage.

Optimization

We designed our chip to be parameterizable on many fronts, because we didn't know beforehand the degree to which it could be scaled. In initial testing of the systolic array we worked with purely 4x4 matrices. However, to show that the Pipette chip was capable of performing complex matrix multiplication we increased this to 8x8 matrices. Initially, we had INT8 source and weight matrices and INT16 accumulate and output, but we found that INT32 output better accommodate the multiplication of INT8 matrices with an added bias.

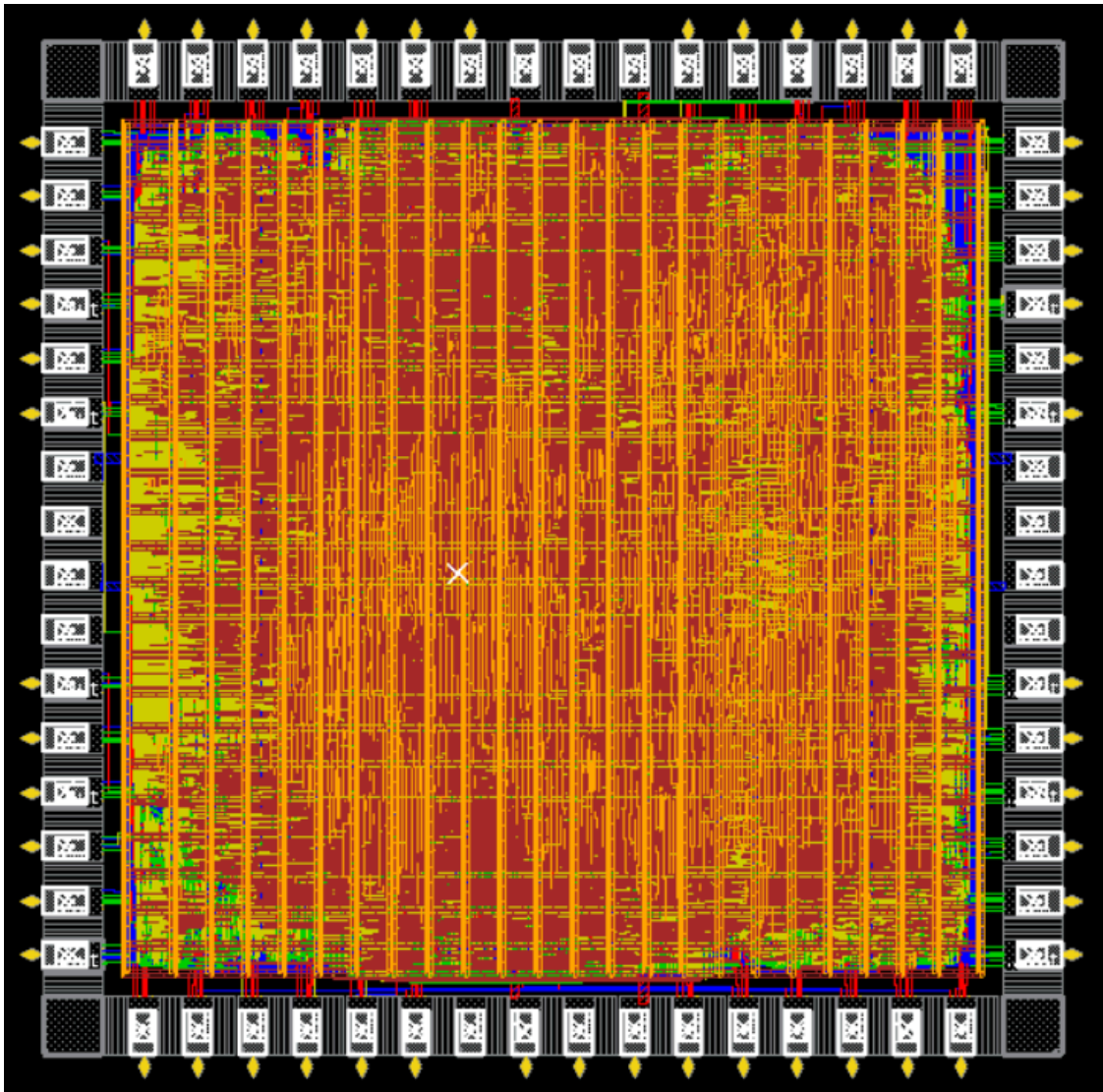
We also spent a lot of time optimizing memory. Initially, we were going to design a memory with multiple banks and a request system. We designed the memory such that both INT8 and INT16 matrices could be stored in any bank (with INT8 taking up half the space). However, we realized that we could predict what kinds of matrices would be stored in the chip. We ended up using a dedicated bank system with a source bank, a weight bank, and an accumulate/result bank. The source and weight bank were combined and could only be written to simultaneously but read from individually (to make computations more efficient). Due to area constraints, we had to use a memory that could hold 4 non-hardcoded source/weight matrices and 2 non-hardcoded accumulate/result matrices.

Lastly, Codesign and optimization of pin layout and floorplanning was crucial for generating layout that satisfies timing constraints while being DRC/LVS clean. The input pins were gathered around top of the chip and outputs of the chip were gathered around bottom, naturally resulting in modules closer to input being floorplanned near the top, and modules closer to output being floorplanned near the bottom. Furthermore, memory being in the center of the chip greatly improved timing analysis results, as most modules and instructions access memory. Small combinational blocks were left un-planned, leaving Innovus to decide microoptimizations.

Evaluation Results

Layout Photo

Figure 5 below shows the full chip with routing and padding in Cadence Innovus. Each color represents a different metal layer, from METAL1 up to METAL6. The white pads on the outside are the I/O pads for external interfacing with the chip.

Figure 5 - Innovus Chip Layout Photo

Cadence Innovus photo of the metal routing used on our chip.

Quality Metrics

The key quality metrics for this design were extracted from post-routing Integrated Circuit (IC) design tools following full parasitic Extraction (RCX) and Static Timing Analysis (STA). To ensure robust in-field reliability, the physical implementation was verified under worst-case operating conditions across distinct process corners:

- Setup timing was verified at the Slow-Slow (SS) process corner, minimum supply voltage, and maximum operating temperature.
- Hold timing was verified at the Fast-Fast (FF) process corner, maximum supply voltage, and minimum operating temperature.

The final layout achieved a highly compact footprint, satisfying all hard physical constraints. The specific structural and timing metrics derived from the post-routing implementation reports are summarized below:

- Final Layout Dimensions: 2.0 mm x 2.0 mm
- Total Silicon Area: 4.0 mm²
- Area Utilization: 76.1%
- Minimum Setup Slack: +75 ps
- Minimum Hold Slack: +124 ps

The hard restriction on physical area was completely satisfied, with an area utilization of 76.1% leaving adequate routing resources to guarantee a DRC-clean layout. The performance target was explicitly bounded at 50 MHz (clock period $T = 20$ ns) because the host FPGA supplies a 50 MHz reference clock, and synthesizing an internal high-speed clock domain beyond this baseline introduces severe I/O interface reliability risks. With positive setup (+75 ps) and hold (+124 ps) slacks, the design successfully achieves timing closure at the required 50 MHz operating frequency.

Furthermore, the reported power dissipation of 115 mW within a 2 mm x 2 mm die size is entirely viable for standard, low-cost plastic packaging (such as a QFP or QFN). Plastic packages typically exhibit a junction-to-ambient thermal resistance (θ_{JA}) ranging from 30°C/W to 50°C/W. A power draw of 115 mW results in a negligible thermal rise of only 3.5°C to 5.8°C above ambient temperature, ensuring the device operates safely well below maximum silicon thermal thresholds without requiring expensive ceramic packaging or external heat sinking.

To evaluate the operational efficiency of the final architecture, a continuous test window was profiled during post-synthesis simulation. The system initiates its first memory transaction at 6,250,000 ps and completes its final transmission at 39,790,000 ps. This yields a total active execution window (Δt) calculated in Equation 1:

$$(1) \Delta t = 39790000ps - 6250000ps = 33540000ps$$

Given the hardware clock period of 20,000 ps (20 ns), the total number of clock cycles spanned during this processing interval, calculated in Equation 2:

$$(2) \frac{33540000ps}{20000ps/cycle} = 1677 \text{ cycles}$$

During this timeframe, the accelerator successfully processed a total workload of 12 distinct macro-operations, consisting of 5 matrix load operations and 7 matrix computation operations. The average architectural efficiency is determined by computing the average cycles consumed per operation. Using the native 50 MHz host clock frequency (50,000,000 cycles/second), the final throughput of the computational design is calculated in Equation 3:

$$(3) \text{Average cycles per operation} = \frac{1677 \text{ cycles}}{12 \text{ operations}} = 139.75 \text{ cycles/operation}$$

This computational rate demonstrates that the underlying hardware successfully balances memory ingestion and parallel execution units, meeting all performance metrics dictated by the system specification.

Analyses

The final SAMMA Pipette design met the primary physical design and performance goals. The chip fit within the $2.0\text{ mm} \times 2.0\text{ mm}$ die constraint and achieved an area utilization of 76.1%. Although this is slightly below the ideal utilization target of 80%, it is a reasonable tradeoff because the remaining whitespace provided routing flexibility and helped achieve a DRC-clean layout. A more aggressive utilization target may have reduced area overhead, but it would have increased routing congestion and made timing closure more difficult.

Timing results were satisfactory. The design closed timing at the target 50 MHz operating frequency, with a minimum setup slack of +75 ps and a minimum hold slack of +124 ps after post-routing extraction and static timing analysis. These positive slack values indicate that the final routed design is timing-clean across the tested process corners. However, the setup slack margin is relatively small, meaning future design changes could easily introduce timing violations. Additional pipeline registers, improved placement of timing-critical modules, and reduced fanout on control signals would improve timing robustness.

The power result was especially strong. The final design dissipated approximately 115 mW, which is below the original exceptional target of 120 mW. This supports the main motivation of the pipette architecture: reducing unnecessary shifting and register overhead compared with a more traditional systolic array implementation. The estimated thermal rise is small enough that the design should be compatible with standard low-cost plastic packaging without requiring external heat sinking.

From a performance perspective, the design completed 12 macro-operations across a 1677-cycle active execution window, corresponding to an average of 139.75 cycles per macro-operation. At the 50 MHz operating frequency, this is approximately 357,800 macro-operations per second. This confirms that the accelerator can operate correctly at the target frequency, but it also shows that memory loading, instruction overhead, and communication latency still contribute significantly to total execution time. Future work should focus on reducing idle cycles between transactions, improving buffering, and increasing systolic array utilization during continuous workloads.

Overall, the final design successfully met the major goals of fitting within the die area, closing timing, maintaining low power, and demonstrating correct accelerator operation. The most important future improvements would be increasing setup timing margin, reducing system-level overhead, and improving sustained throughput for larger matrix workloads.

References

- [1] J. P. de Gyvez, "Yield modeling: An engineering approach," *IEEE Transactions on Semiconductor Manufacturing*, vol. 19, no. 3, pp. 295–299, Aug. 2006.
- [2] B. Razavi, "Design of phase-locked loops for high-speed circuits," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2003, pp. IV-900–IV-903.
- [3] IEEE Std 1149.1-2013, *IEEE Standard for Test Access Port and Boundary-Scan Architecture*, IEEE, May 13, 2013. Accessed: Jun. 5, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/6515989>
- [4] A. J. Abdelmaksoud, S. Agwa, and T. Prodromakis, "DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration," *IEEE Trans. Circuits Syst. I*, vol. 73, no. 1, pp. 283–293, Jan. 2026, doi: 10.1109/TCSI.2025.3591960.
- [5] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," Apr. 16, 2017, *arXiv*: arXiv:1704.04760. doi: 10.48550/arXiv.1704.04760
- [6] Kung, "Why systolic architectures?," *Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982, doi: 10.1109/MC.1982.1653825.
- [7] H. Genc *et al.*, "Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, San Francisco, CA, USA: IEEE, Dec. 2021, pp. 769–774. doi: 10.1109/DAC18074.2021.9586216.
- [8] Texas Instruments, *Serial Peripheral Interface (SPI) for KeyStone Devices User's Guide*, Rev. A, SPRUGP2A, Mar. 2012. Accessed: Jun. 5, 2026. [Online]. Available: <https://www.ti.com/lit/ug/sprugp2a/sprugp2a.pdf>
- [9] A. Negi, "Arduino Due Pinout, Specifications, Schematic & Datasheet," *eTechnophiles*, Aug. 5, 2022. Accessed: Jun. 7, 2026. [Online]. Available: <https://www.etechnophiles.com/arduino-due-pinout-specifications-schematic-datasheet>